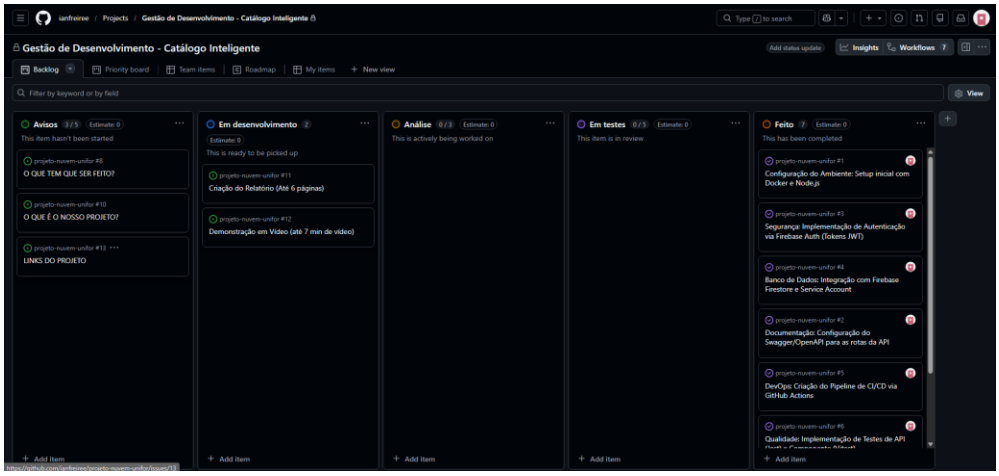
RELATÓRIO TÉCNICO: SISTEMA DE CATÁLOGO INTELIGENTE EM NUVEM

Disciplina: Desenvolvimento de Software em Nuvem – UNIFOR

Equipe: Ian Aureliano Freire, Henri Aureliano Freire

1. INTRODUÇÃO E OBJETIVO

Este documento detalha a implementação de um sistema de gerenciamento de estoque baseado em arquitetura de microserviços e computação em nuvem. O objetivo foi aplicar conceitos de **IaaS (Infrastructure as a Service)** e **PaaS (Platform as a Service)** para criar uma aplicação escalável, segura e com deploy automatizado.

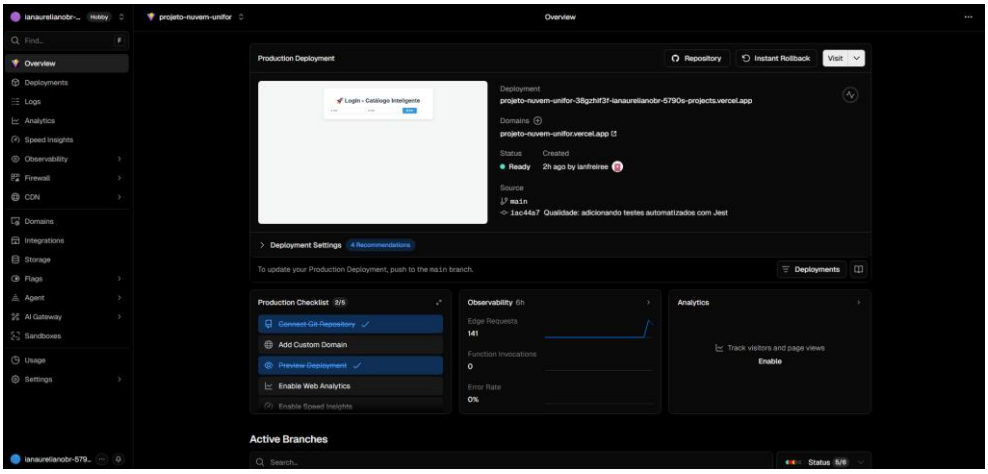


Legenda: Organização do fluxo de trabalho utilizando Kanban no GitHub Projects para gestão de tarefas e prazos.

2. ARQUITETURA DO SISTEMA

A solução foi desenhada para ser totalmente desacoplada, utilizando as melhores práticas de mercado:

- **Frontend (React):** Uma interface reativa hospedada na **Vercel**, focada na experiência do usuário e comunicação via API.
- **Backend (Node.js):** API RESTful containerizada com **Docker**, responsável pela lógica de negócio e segurança.
- **Banco de Dados (Firestore):** Banco de dados NoSQL orientado a documentos para persistência elástica.

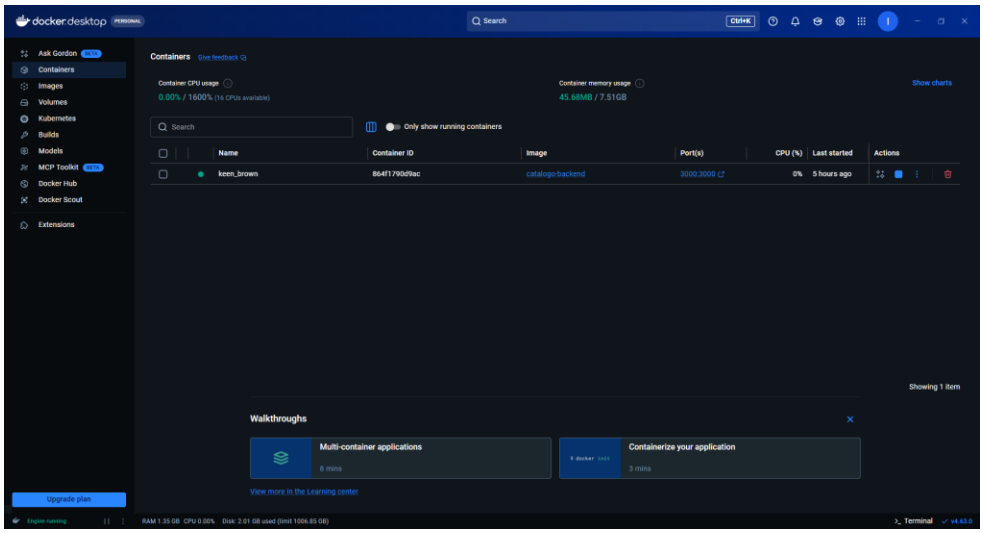


Legenda: Interface do usuário final em ambiente de produção (Vercel) e status de deploy da camada cliente.

3. INFRAESTRUTURA COMO CÓDIGO E CONTAINERIZAÇÃO (DOCKER)

Para garantir que a aplicação rode de forma idêntica em qualquer ambiente, utilizamos o Docker. O backend foi isolado em uma imagem imutável, eliminando o clássico problema do "na minha máquina funciona".

- **Dockerfile:** Configurado para expor a porta 3000 e otimizado para produção.
- **Isolamento:** As dependências locais (`node_modules`) são ignoradas, sendo instaladas apenas dentro do container.

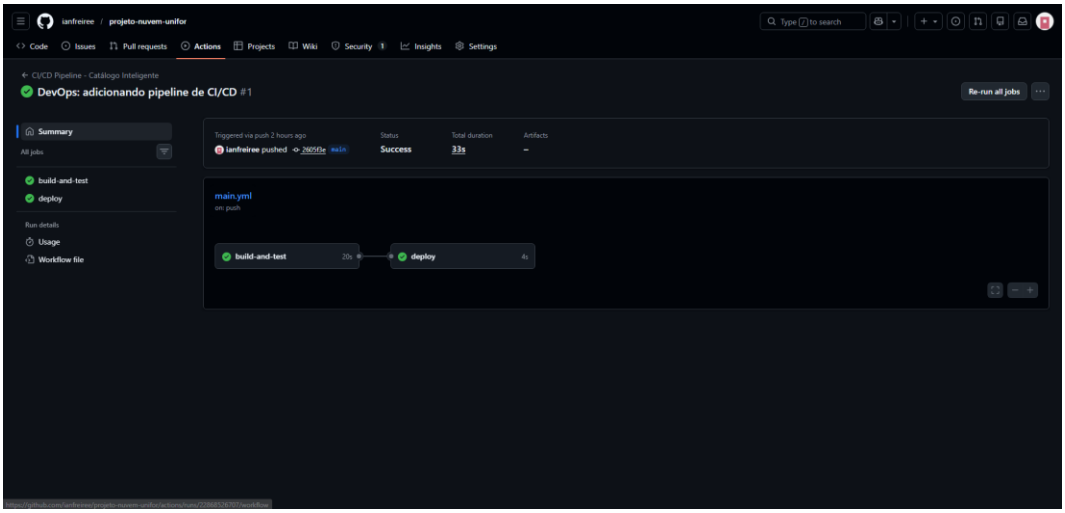


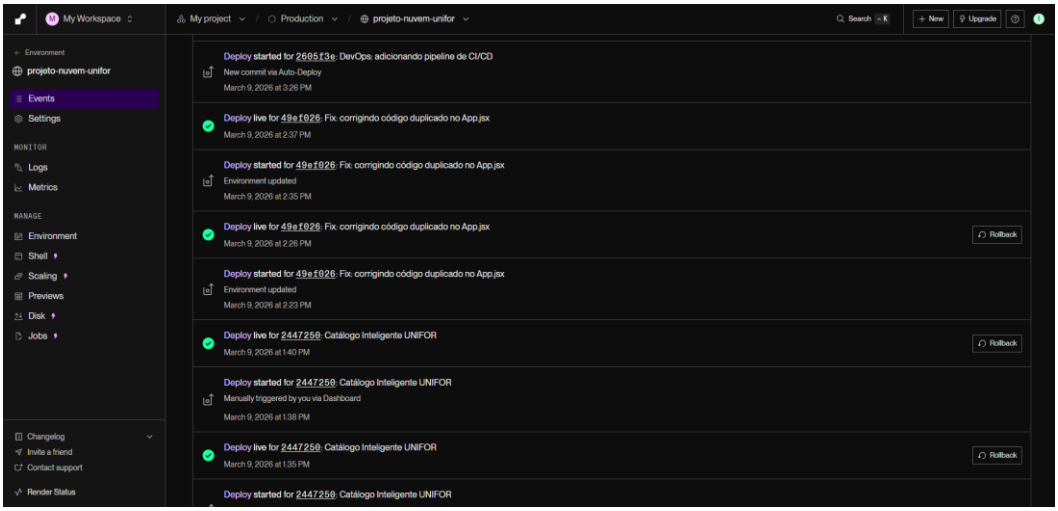
Legenda: Definição do Dockerfile e processos de build da imagem do servidor backend.

4. ESTRATÉGIA DE DEPLOY E CI/CD

Implementamos um pipeline de **Integração e Entrega Contínua (CI/CD)**. Qualquer alteração na branch `main` dispara automaticamente um novo build no Render e na Vercel.

- **Render:** Hospeda o container Docker. Utilizamos **Secret Files** para injetar as chaves do Firebase de forma segura, sem expô-las no GitHub.
- **GitHub Actions:** Automação de testes e verificações de qualidade antes do deploy.



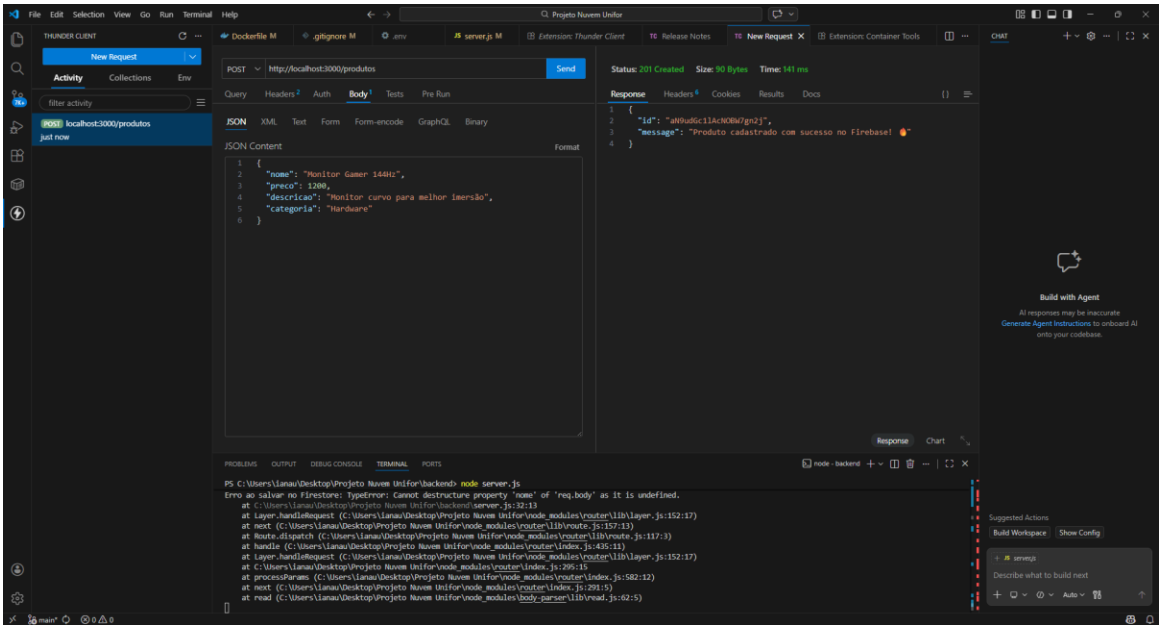


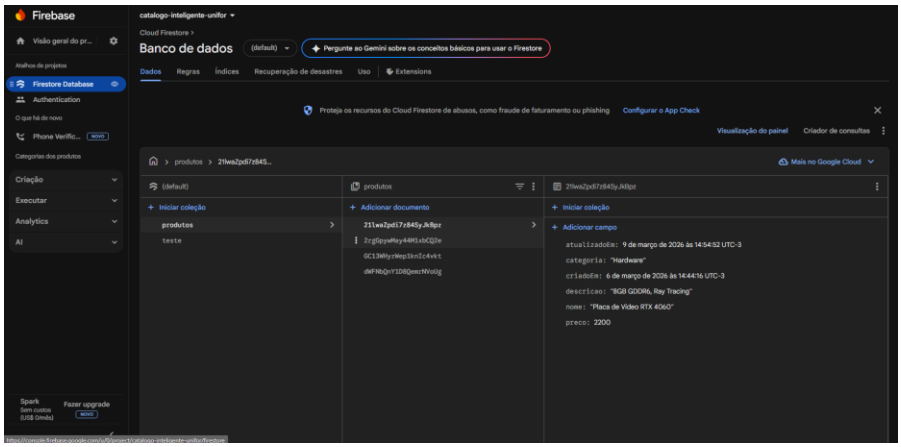
Legenda: Fluxo de automação do GitHub Actions e logs de inicialização bem-sucedida no Render.

5. PERSISTÊNCIA E SEGURANÇA (FIREBASE & JWT)

A segurança é garantida por tokens **JWT (JSON Web Tokens)** via Firebase Auth.

1. O usuário faz login no frontend.
2. Um token é gerado e enviado no cabeçalho `Authorization: Bearer`.
3. O servidor valida o token antes de permitir qualquer operação de escrita (POST, PUT, DELETE) no Firestore.



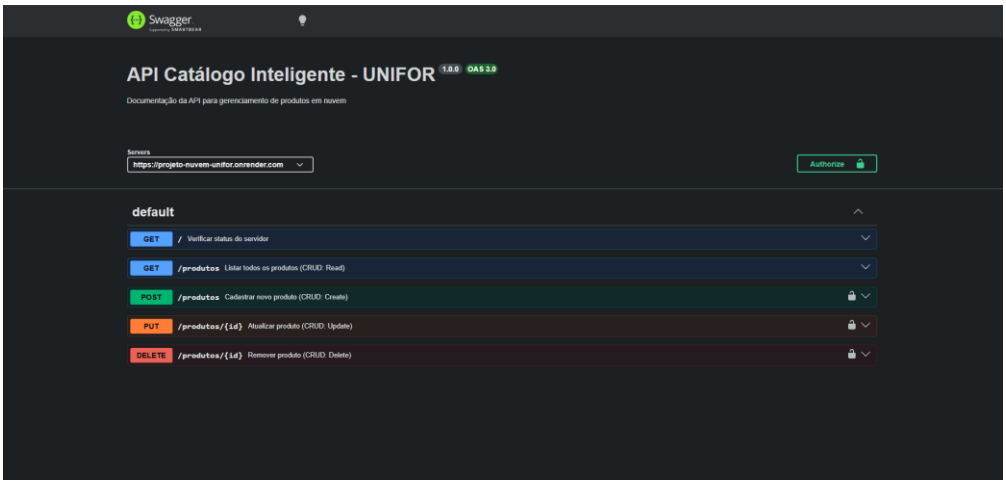


Legenda: Validação da persistência de dados no console do Google Cloud (Firestore) após operação de cadastro.

6. DOCUMENTAÇÃO E INTEROPERABILIDADE (SWAGGER)

Seguindo o padrão **OpenAPI 3.0**, a API foi totalmente documentada. Isso permite que outros desenvolvedores ou sistemas consumam nossos serviços de forma clara e padronizada.

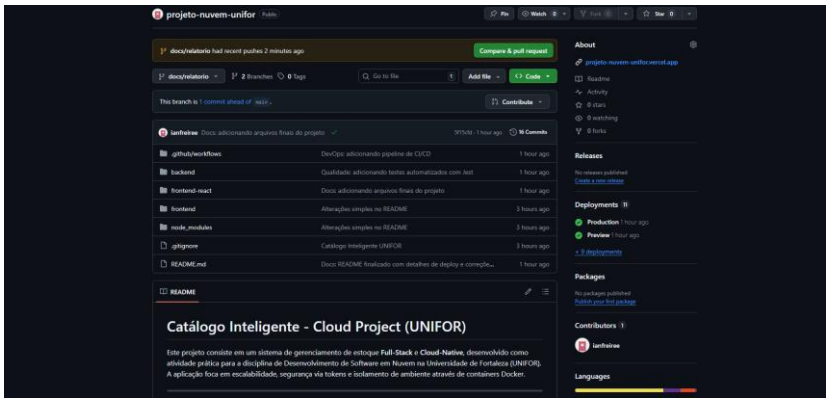
- **Endpoint:** `/api-docs`
- **Segurança:** Implementação de suporte a Bearer Auth diretamente na interface do Swagger.

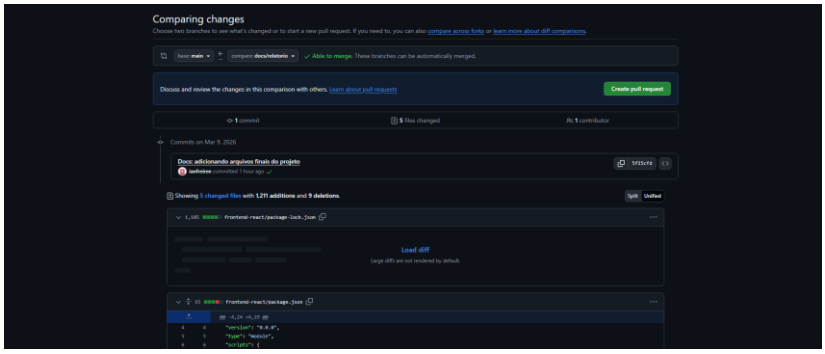


7. GESTÃO DE VERSIONAMENTO E BRANCHES

Utilizamos uma estratégia de branches para organizar o desenvolvimento:

- **main:** Código estável em produção.
- **docs/relatorio:** Isolamento da documentação técnica e relatórios de evidências.





Legenda: Estrutura de branches no GitHub garantindo a integridade do código fonte e organização da documentação.

8. CONCLUSÃO

O projeto demonstrou a eficiência de arquiteturas modernas em nuvem. Superamos desafios críticos como a gestão de CORS e a configuração de portas em containers Docker. O resultado final é um sistema robusto, com documentação automática e deploy automatizado, pronto para escala real.

9. LINKS UTEIS:

Link do Repositório (GitHub): https://github.com/ianfreiree/projeto-nuvem-unifor.git
Link do Sistema (Vercel): https://projeto-nuvem-unifor.vercel.app/

Link da Documentação (Swagger): https://projeto-nuvem-unifor.onrender.com/api-docs